CYBERSECURITY LAB REPORT

Report #9 — OWASP Juice Shop: Advanced Web Exploitation

Target: http://192.168.255.128:8085

Attacker: Kali Linux (Burp Suite Community Edition v2026.3.2)

Date: June 2026

Techniques: Burp Suite Setup • CSRF PoC • Broken Access Control • SQL Injection
Authentication Bypass • NoSQL Injection • Steganography

1. Introduction

This report documents the ninth phase of the structured OWASP Juice Shop penetration testing laboratory series. In previous reports, reconnaissance, vulnerability scanning, SMB exploitation, Log4Shell, and an initial round of Juice Shop web-application attacks were performed. Report 9 focuses on a more advanced exploitation session executed entirely through Burp Suite Community Edition v2026.3.2, covering authentication bypass, Cross-Site Request Forgery (CSRF), Broken Access Control via API manipulation, SQL Injection-based account takeover, NoSQL injection, and steganographic data extraction.

A key objective of this session was to go beyond automated tooling and build manual, working proof-of-concept (PoC) exploits for each vulnerability class. Automated scanners frequently produce false negatives on client-side flaws such as CSRF or miss chained attack paths entirely; this lab demonstrates the value of manual verification and PoC construction. I also worked through Juice Shop's Coding Challenges to reinforce the link between exploited behavior and vulnerable source code.

All attacks were conducted in an isolated Docker-based lab environment. The Juice Shop instance ran at http://192.168.255.128:8085 and no production systems were involved. Findings are accompanied by remediation guidance aligned with OWASP Top 10 (2021) and secure development best practices.

Scope: OWASP Juice Shop v15.x on Docker (192.168.255.128:8085).

Attacker machine: Kali Linux with Burp Suite Community Edition v2026.3.2.

Challenge progress at session end: 62/178 challenges solved (30% Hacking, 42% Coding). Final session total: 100/178 (54% Hacking, 61% Coding).

2. Environment Setup: Burp Suite Installation & Proxy Configuration

2.1 Burp Suite Installation

Burp Suite Community Edition v2026.3.2 was not pre-installed in I's Kali instance and had to be obtained and installed via apt. The installation required OpenJDK 25 JRE as a dependency, which apt resolved and unpacked automatically alongside the burpsuite package itself.

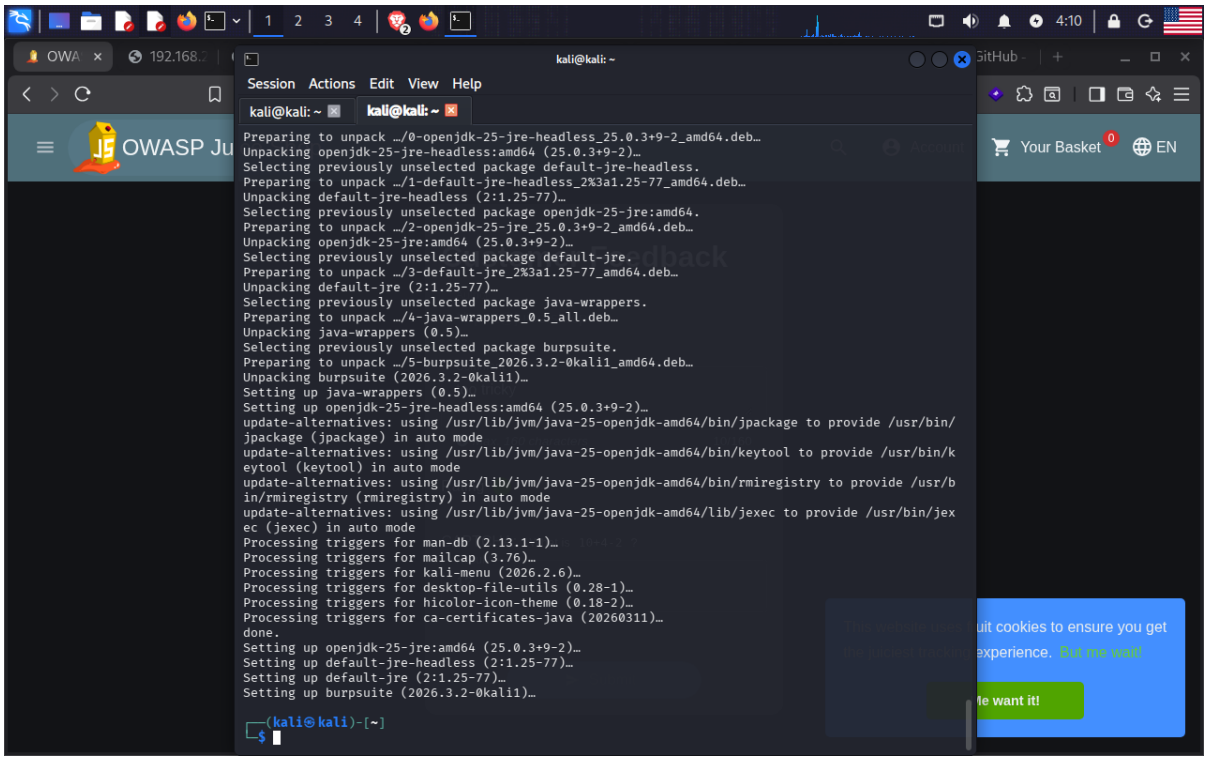

Figure 1. Terminal output of apt installing Burp Suite Community Edition v2026.3.2 on Kali Linux. The dependency chain includes openjdk-25-jre-headless, default-jre-headless, openjdk-25-jre, and java-wrappers before the burpsuite package itself is configured. Installation completes successfully, as indicated by the final "done." message and "Setting up burpsuite (2026.3.2-0kali1)" line.

2.2 Proxy Listener Configuration

After launching Burp Suite from the ~/Desktop/evidencia directory with the burpsuite command, the Proxy settings panel was opened to verify the listener address. The default listener was confirmed active on 127.0.0.1:8080 with the "Running" checkbox enabled. The browser was subsequently configured to route all HTTP traffic through this address so that Burp Suite could intercept, inspect, and replay every request sent to the Juice Shop target.

Establishing an intercepting proxy between the browser and the target application is a prerequisite for all manual web-application testing. It allows I to inspect raw HTTP traffic, modify request parameters in transit, replay requests with altered payloads, and examine server responses in detail — capabilities that no passive scanner can replicate.

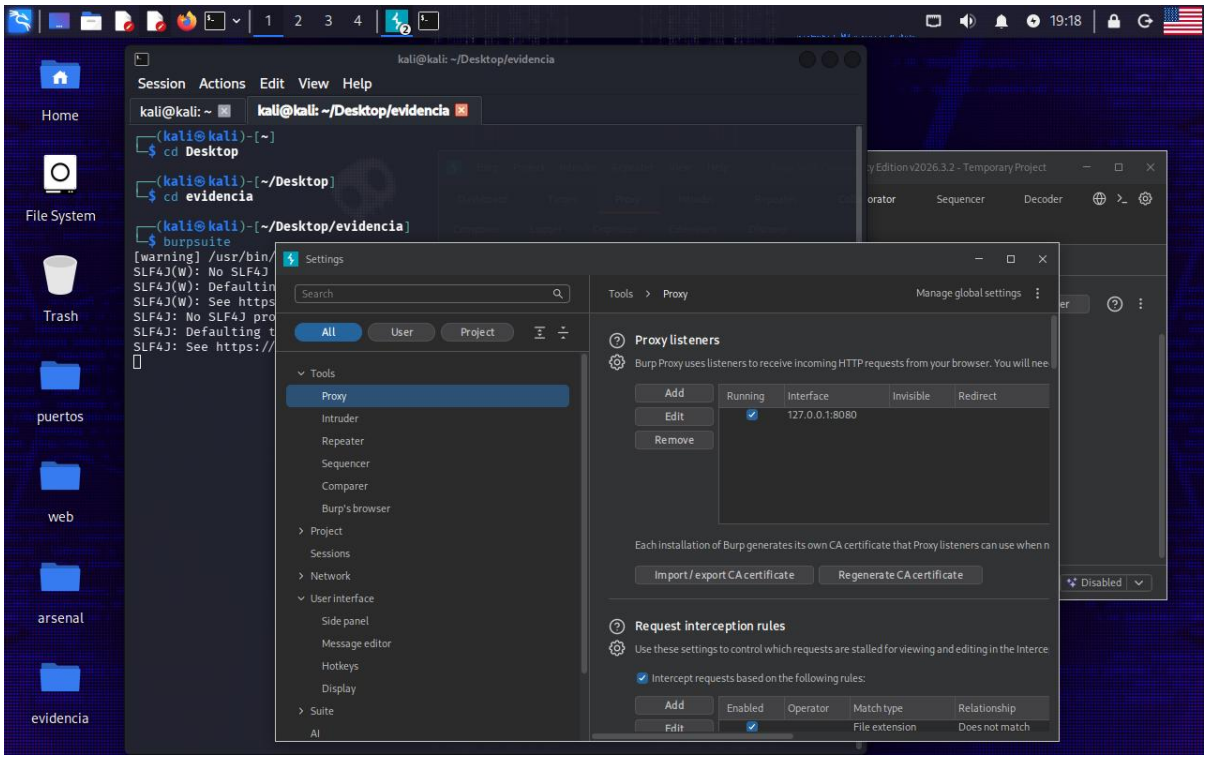

Figure 2. Burp Suite Community Edition v2026.3.2 Proxy settings panel, launched from ~/Desktop/evidencia via the burpsuite command. The Proxy Listener table confirms an active listener on 127.0.0.1:8080 (Running checkbox enabled). Request interception rules are visible in the lower pane, set to "Intercept requests based on the following rules." SLF4J warnings in the background terminal are benign logging framework initialization messages.

3. Score Board & Traffic Reconnaissance

Before executing any attacks, the Juice Shop Score Board was located by navigating to the hidden route /api/challenges or by inspecting the application's JavaScript bundles — a one-star "Security through Obscurity" challenge in itself. The Score Board provides a real-time view of every available challenge, categorized by vulnerability class and difficulty, making it the central dashboard for guiding the testing session.

Simultaneously, Burp Suite's HTTP History tab was open to capture and analyze all background WebSocket polling traffic from the Angular front end (socket.io?EIO=4&transport=polling). Reviewing this traffic established the application's API surface and confirmed that all session tokens were being transmitted via JWT cookies, which would be relevant for later authentication bypass attempts.

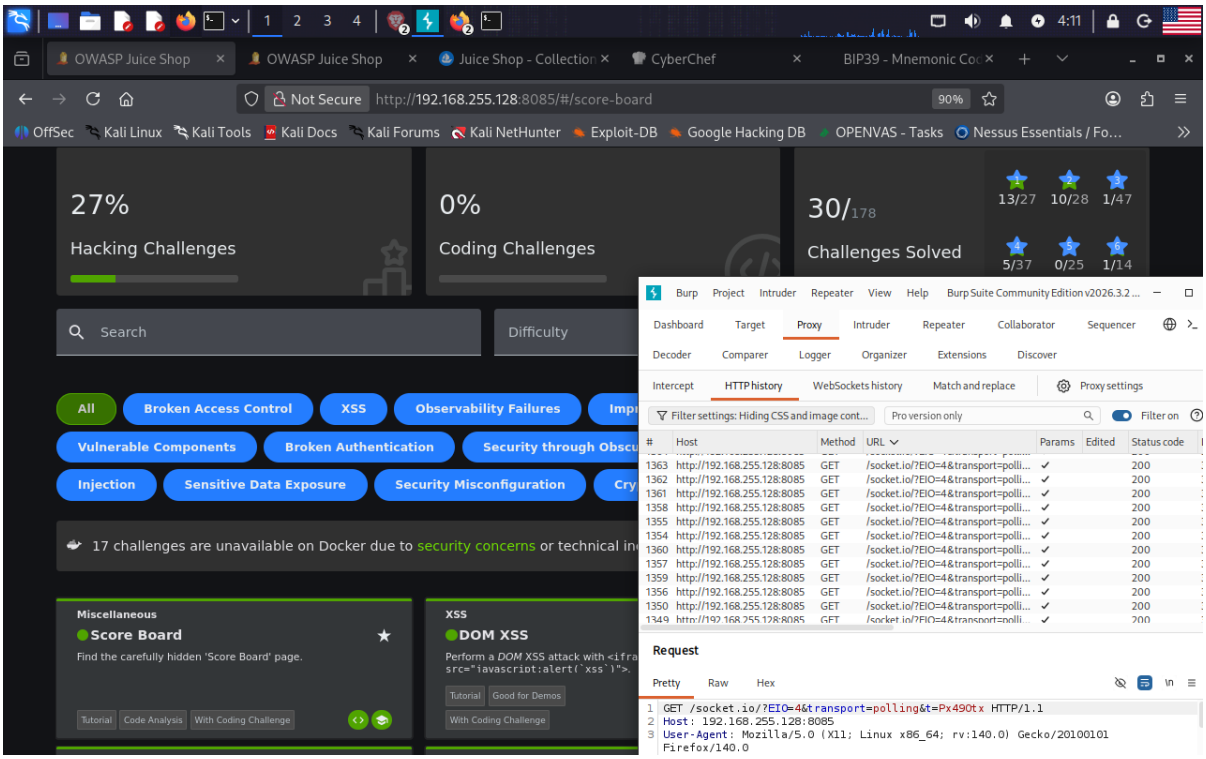

Figure 3. Firefox browser displaying the OWASP Juice Shop Score Board at http://192.168.255.128:8085/#/score-board, overlaid with the Burp Suite HTTP History tab. The Score Board shows 30/178 total challenges solved at this point (13/27 one-star, 10/28 two-star, 1/47 three-star). Challenge categories visible include Broken Access Control, XSS, Observability Failures, Improper Input Validation, Unvalidated Redirects, Vulnerable Components, Broken Authentication, Security through Obscurity, Injection, Sensitive Data Exposure, and Security Misconfiguration. Burp Suite HTTP History captures WebSocket long-polling requests (GET /socket.io?EIO=4&transport=polling) returning HTTP 200, confirming live traffic interception on port 8085.

4. Challenge Progress Overview

Figure 4 presents the Score Board state after a significant portion of the session was complete, reflecting 62 out of 178 total challenges solved. At this point, Hacking Challenges stood at 30% and Coding Challenges at 42%, indicating that I was actively cross-referencing exploited behavior with the corresponding vulnerable source code visible in the Coding Challenge section. This deliberate practice — exploiting a vulnerability and then locating the exact server-side code that permits it — is central to secure code review skills required in AppSec and GRC roles.

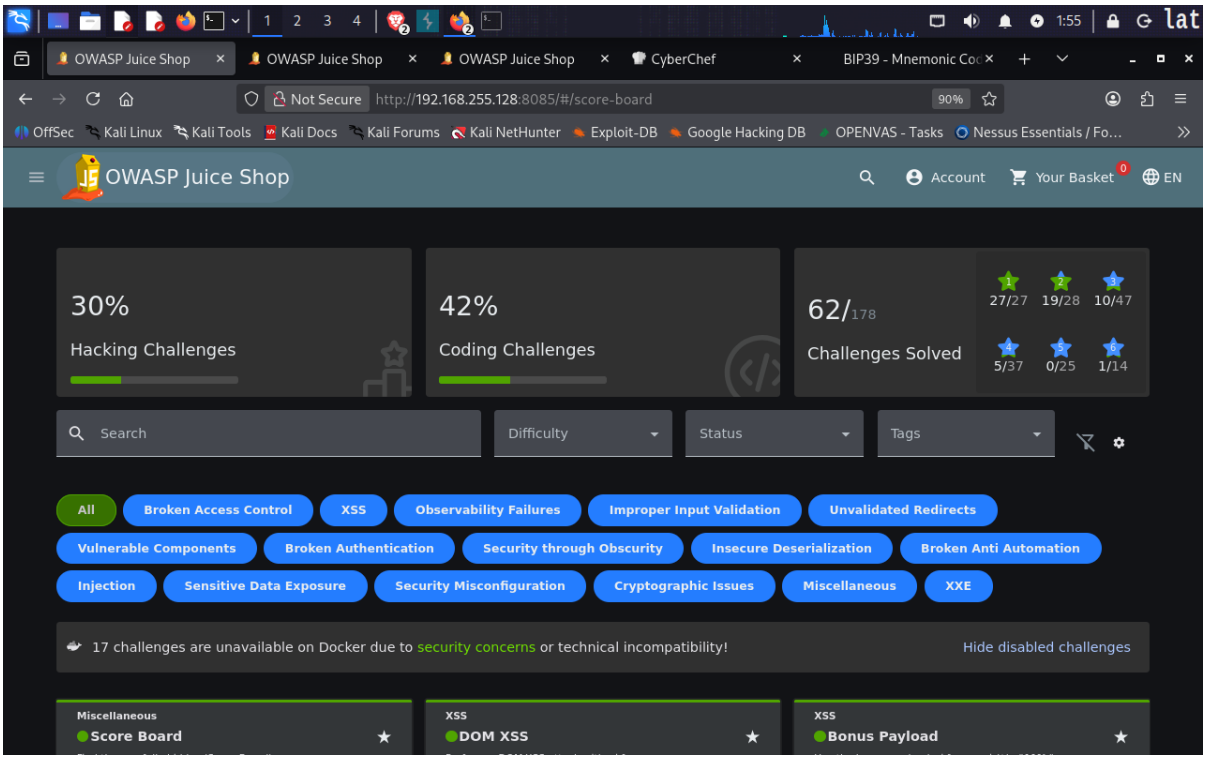

Figure 4. OWASP Juice Shop Score Board showing mid-session challenge progress: 62/178 challenges solved, 30% Hacking Challenges, 42% Coding Challenges. Star-rating breakdown: 27/27 (1-star), 19/28 (2-star), 10/47 (3-star), 5/37 (4-star), 0/25 (5-star), 1/14 (6-star). Category tags visible include Broken Access Control, XSS, Observability Failures, Improper Input Validation, Unvalidated Redirects, Vulnerable Components, Broken Authentication, Security through Obscurity, Insecure Deserialization, Broken Anti Automation, Injection, Sensitive Data Exposure, Security Misconfiguration, Cryptographic Issues, Miscellaneous, and XXE. A banner notes that 17 challenges are unavailable on Docker.

5. Cross-Site Request Forgery (CSRF) — Manual PoC Construction

5.1 Vulnerability Overview

Cross-Site Request Forgery (CSRF) abuses the browser's automatic inclusion of session cookies in cross-origin requests. If a web application does not validate the origin of a state-changing request (e.g., via a CSRF token, SameSite cookie attribute, or Origin header check), a malicious page hosted on a different domain can silently trigger actions on behalf of an authenticated victim.

Automated scanners frequently miss CSRF vulnerabilities because they cannot simulate an authenticated session across origins. This exercise demonstrates the superior coverage of manual PoC construction: a working exploit was crafted and executed despite the Score Board categorizing the Juice Shop as not having detected the attack.

5.2 PoC HTML Construction

A malicious HTML form was created targeting the Juice Shop profile endpoint (http://192.168.255.128:8085/profile) using the POST method. The form contained a hidden input field pre-populating the username parameter with the value "CSRF_Success_1337" — a sentinel value chosen to confirm that the cross-origin write succeeded. A JavaScript snippet was added to auto-submit the

form on page load, simulating a victim clicking a malicious link. The form was staged using the real-time HTML editor at htmledit.squarefree.com to simulate cross-origin hosting.

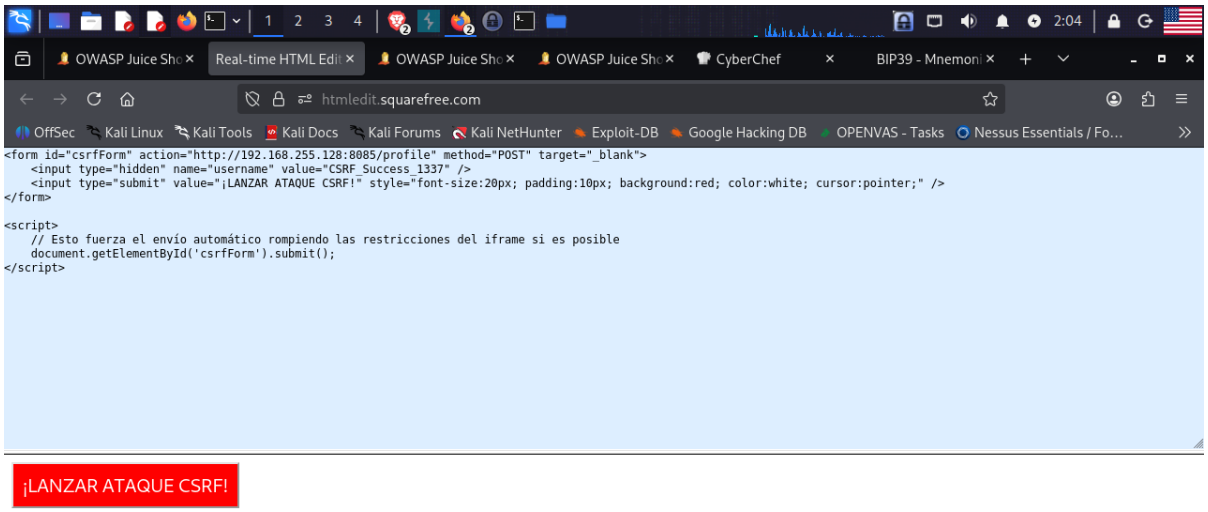

Figure 5. Real-time HTML editor at htmledit.squarefree.com displaying the CSRF proof-of-concept payload. The form targets http://192.168.255.128:8085/profile via POST method with target="_blank". A hidden input sets the username field to "CSRF_Success_1337". A <script> block calls document.getElementById('csrfForm').submit() automatically on load, forcing the cross-origin request without user interaction. The red button labeled "LANZAR ATAQUE CSRF!" confirms the rendered output and successful form construction.

5.3 Exploitation Result

Upon executing the PoC, the browser navigated to the Juice Shop User Profile page and confirmed that the username field had been updated to "CSRF_Success_1337" — visible both in the Username input box and as a display name below the avatar image. The email shown (support@juice-sh.op) confirms the request was executed in the context of an authenticated session, validating the CSRF exploit.

This outcome demonstrates the CSRF vulnerability in the /profile endpoint: the server accepted the POST request from a cross-origin form without validating a CSRF token or enforcing SameSite cookie restrictions. The challenge was solved by proving the state-changing write succeeded.

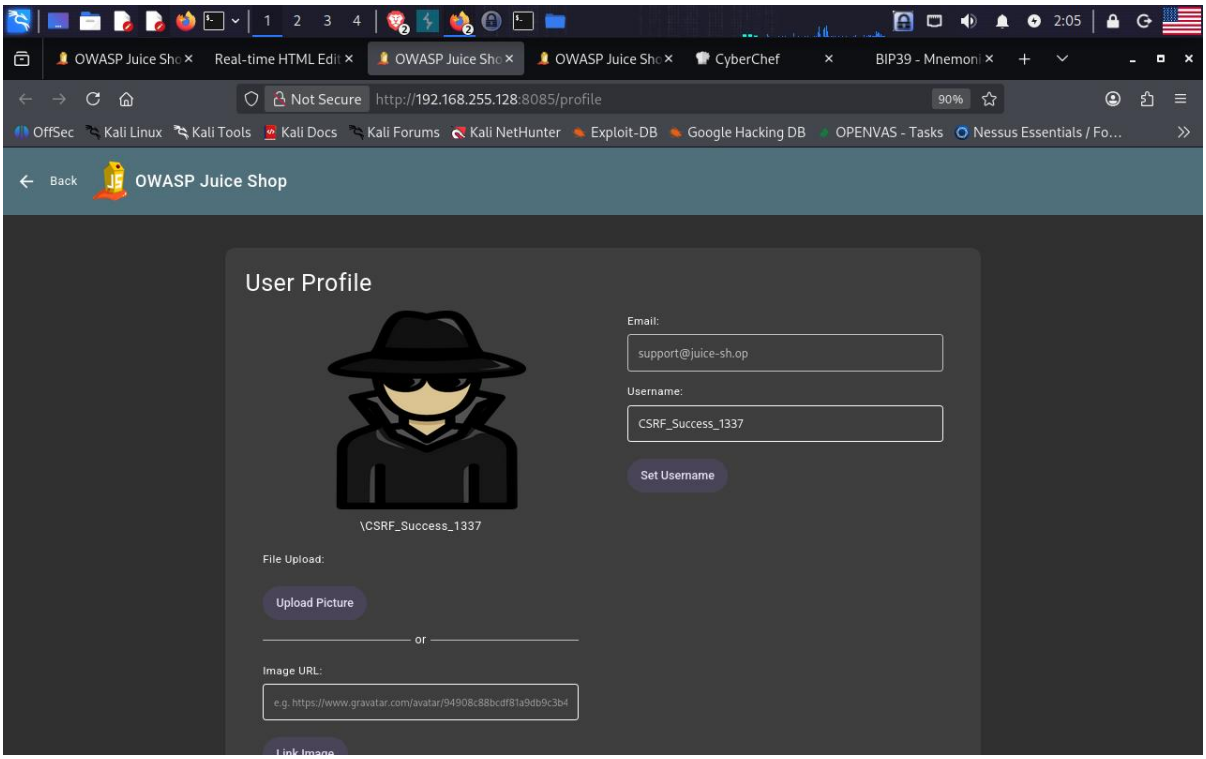

Figure 6. OWASP Juice Shop User Profile page at http://192.168.255.128:8085/profile after CSRF exploit execution. The Username field displays "CSRF_Success_1337" and the display name below the avatar confirms the same value, proving the cross-origin POST to /profile successfully updated the account's username. The authenticated session (email: support@juice-sh.op) was used without the user's knowledge or consent, validating the CSRF vulnerability.

5.4 Remediation

To eliminate CSRF risk, the application should implement the following controls:

- Synchronizer Token Pattern: Generate a unique, unpredictable CSRF token per session and require its presence in all state-changing POST, PUT, PATCH, and DELETE requests. Validate server-side on every submission.
- SameSite Cookie Attribute: Set session cookies with SameSite=Strict or SameSite=Lax. This prevents browsers from including the cookie in cross-site requests, blocking CSRF at the browser level.
- Origin / Referer Header Validation: Reject requests where the Origin or Referer header does not match the application's own origin.
- Double Submit Cookie Pattern: As an alternative stateless approach, include the CSRF token as both a cookie and a request parameter, validating that both values match.

6. Broken Access Control — Unauthorized Deluxe Membership Acquisition

6.1 Vulnerability Overview

Broken Access Control occurs when an application does not properly enforce restrictions on what authenticated users can do. In this case, the Juice Shop's payment and membership upgrade endpoint

(/rest/deluxe-membership) performed no server-side price or authorization validation. By inspecting the API call triggered by the "Become a Deluxe Member" UI flow and modifying the paymentMode parameter, an attacker could complete the upgrade without a valid payment method.

6.2 Attack Execution

Using the Burp Suite Repeater (Tab 1), the PUT request to /rest/deluxe-membership was intercepted and modified. The request body included a paymentMode field set to an empty string ("") and a paymentId of 7 — values that do not correspond to a valid payment instrument. Despite these invalid parameters, the server returned HTTP 200 OK and responded with a JSON body containing status: "success" and a confirmation message: "Congratulations! You are now a deluxe member!" alongside a refreshed JWT in the token field.

This confirms that the authorization boundary was enforced solely by the client-side UI. The server never validated that a real payment had been made or that the paymentId referenced an existing, authorized payment method linked to the user's account.

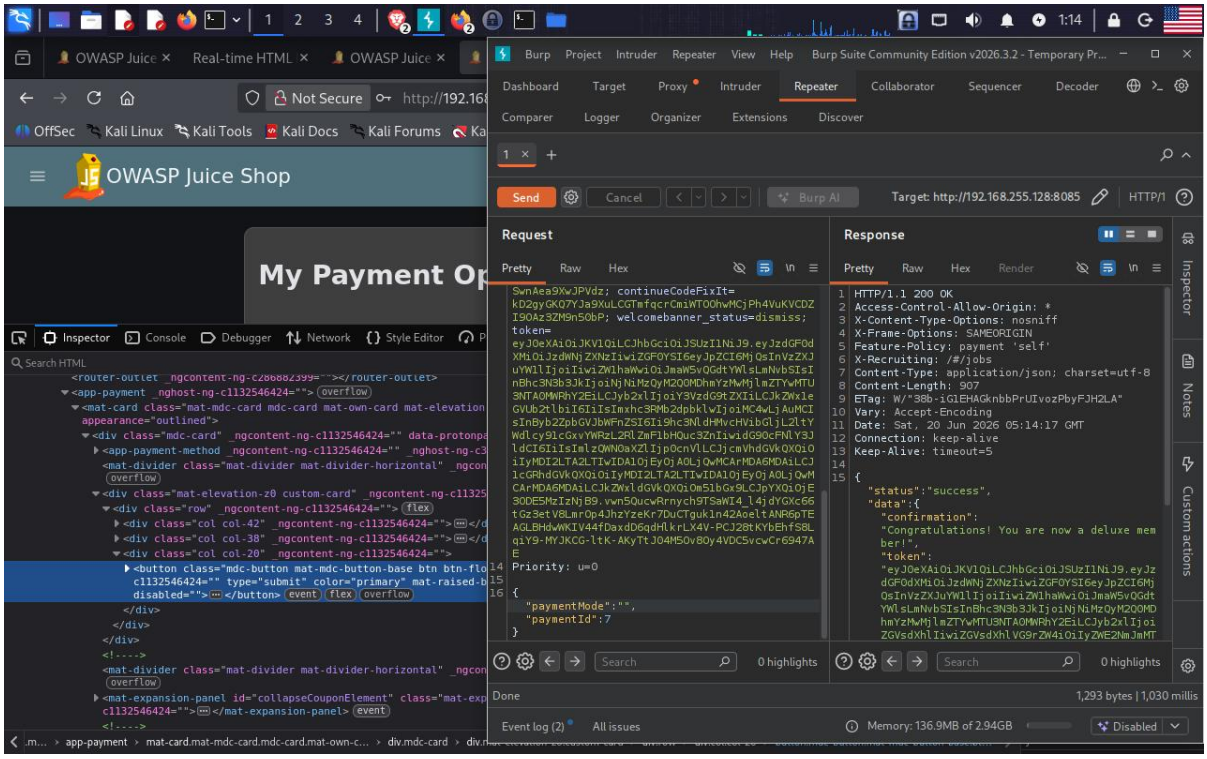

Figure 7. Burp Suite Repeater (Tab 1) showing the PUT request to http://192.168.255.128:8085/rest/deluxe-membership with request body {"paymentMode":"","paymentId":7}. The response (HTTP 1.1 200 OK, Content-Length: 907) returns a JSON body with status: "success" and a confirmation string "Congratulations! You are now a deluxe member!". A refreshed JWT token is included in the response data field. Browser DevTools Inspector visible in the background shows the Angular payment component DOM structure, including the disabled submit button that client-side logic was supposed to enforce.

6.3 Remediation

Server-side authorization must be enforced independently of client-side UI state:

- Validate paymentId server-side: Before processing any membership or payment action, verify that the paymentId exists in the database, belongs to the requesting user, and has sufficient balance or is a valid card on file.

- Reject empty or null paymentMode: The application must treat an empty paymentMode as an invalid state and return HTTP 400/403, not 200.
- Principle of Least Privilege on endpoints: The /rest/deluxe-membership endpoint should require a server-side transaction confirmation, not merely accept client-supplied parameters at face value.
- Security audit of all privileged state-change endpoints: Any endpoint that grants elevated privileges (membership upgrade, role change, credit addition) must have integration tests that attempt invalid inputs and verify that they are rejected.

7. Authentication — Credential Brute-Force with Burp Suite Intruder

7.1 Vulnerability Overview

The "Login Amy" challenge required recovering the password of the account amy@juice-sh.op. The application lacked effective brute-force protections — no account lockout policy, no CAPTCHA, and no rate limiting on the /rest/user/login endpoint. This allowed an automated credential-stuffing or dictionary attack via Burp Suite Intruder to iterate through a password wordlist until the correct credentials were found.

7.2 Attack Execution via Burp Suite Repeater & Intruder

The login request (POST /rest/user/login) was captured in Burp Suite and forwarded to Repeater. A baseline request was crafted with email: "amy@juice-sh.op" and a placeholder password. The request was then sent to Intruder with a Cluster Bomb or Sniper attack type, with the password field marked as a payload position. A relevant wordlist was loaded, and the attack was launched.

Intruder cycled through candidates and identified the correct password when request #130 (Payload 1: "K", with the full constructed value visible as "Klf..." in the Inspector panel) returned HTTP 200 with a Content-Length of 1179 — significantly larger than the 401 responses (length 413) for failed attempts. The successful response body contained a valid JWT authentication token, confirming login.

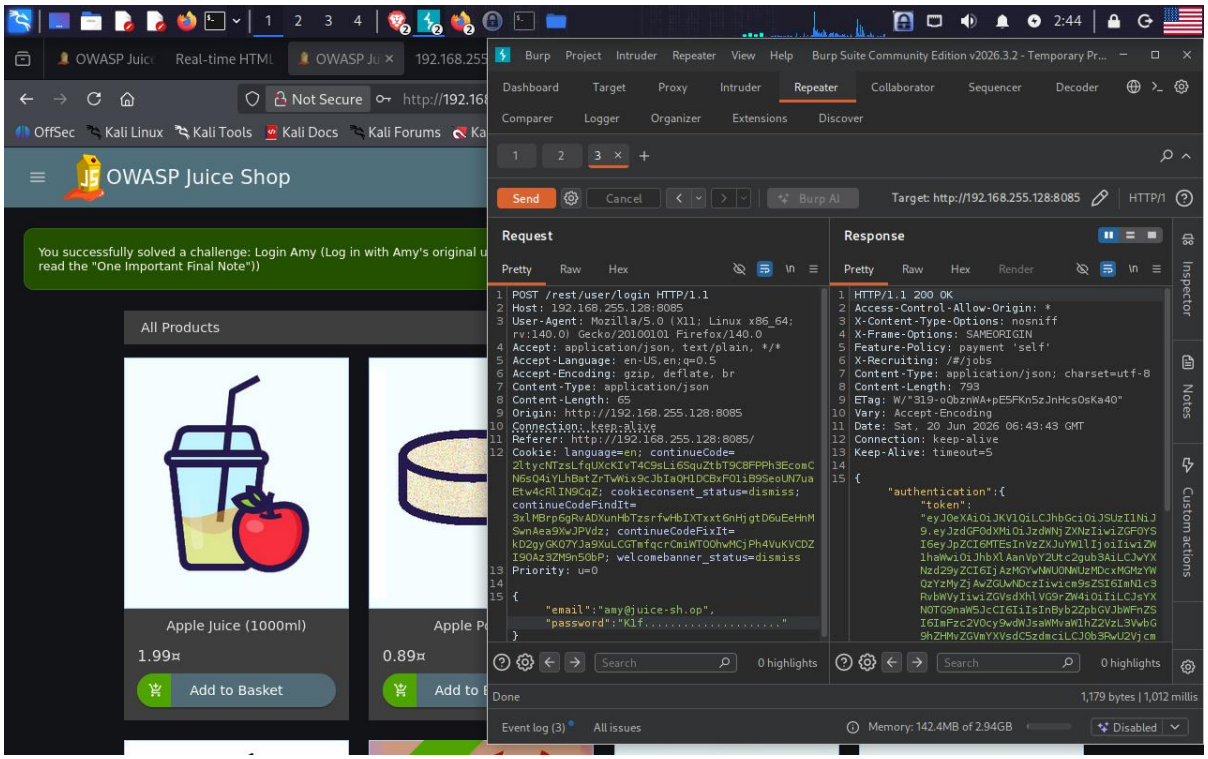

Figure 8. Burp Suite Repeater (Tab 3) confirming successful login for amy@juice-sh.op. The POST /rest/user/login request body includes the email address and the discovered password ("Klf....."). The HTTP 1.1 200 OK response (Content-Length: 793) returns a JSON body with an authentication object containing a valid JWT token. A green Juice Shop notification is visible in the background: "You successfully solved a challenge: Login Amy (Log in with Amy's original username and password)."

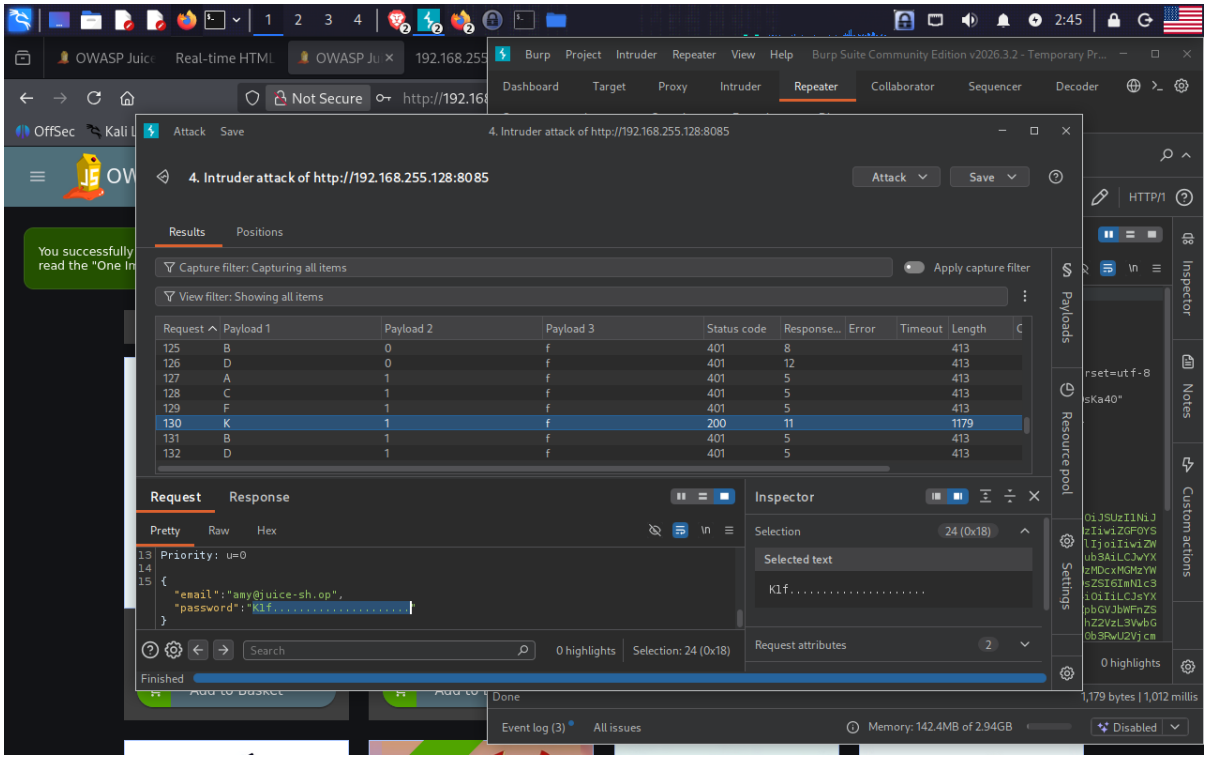

Figure 9. Burp Suite Intruder attack results window ("4. Intruder attack of http://192.168.255.128:8085") showing the credential brute-force in progress. Request #130 (highlighted in blue) returned HTTP 200 with a Response length of 1179 — the only

success among surrounding 401 responses (length 413). The Payload 1 column shows "K" and the Inspector panel (right) displays the Selected Text "Klf.....", confirming the correct password candidate. All other requests returned 401 Unauthorized, demonstrating effective password discovery via differential response analysis.

7.3 Remediation

The following controls should be applied to all authentication endpoints:

- Account lockout / progressive delay: After a configurable number of failed attempts (e.g., 5), temporarily lock the account or introduce exponential back-off delays.
- Rate limiting: Enforce a per-IP and per-account request rate limit on /rest/user/login. Return HTTP 429 Too Many Requests when the threshold is exceeded.
- CAPTCHA or MFA: Require a CAPTCHA after a small number of failures, or enforce multi-factor authentication for all accounts.
- Consistent error responses: Return the same error message and response time regardless of whether the email exists, to prevent user enumeration.
- Monitoring and alerting: Log all failed authentication attempts with IP, timestamp, and account targeted. Alert on anomalous patterns.

8. Broken Access Control — Unauthorized Basket Manipulation

8.1 Vulnerability Overview

The "Manipulate Basket" challenge exposed an Insecure Direct Object Reference (IDOR) vulnerability in the basket item API. The /rest/BasketItems endpoint accepted a BasketId parameter in the request body without verifying that it matched the authenticated user's own basket. By supplying a foreign BasketId in the PUT /rest/BasketItems request, I was able to add a product to another user's basket without any ownership check.

8.2 Attack Execution

Using Burp Suite Repeater (Tab 6), a PUT request to /rest/BasketItems was sent with the following body: {"ProductId":42,"BasketId":"4","quantity":1,"BasketId":"2"}. The JSON object contains a duplicate BasketId key — the first instance ("4") represents the attacker's own basket, while the second ("2") is the target victim basket. Node.js (the Juice Shop backend) resolves duplicate JSON keys by using the last occurrence, causing the item to be inserted into BasketId 2 (the victim's basket).

The server returned HTTP 200 OK with a response body confirming the basket item was created with id:11, ProductId:42, BasketId:"2", quantity:1 — proving the write was performed into the foreign basket without authorization.

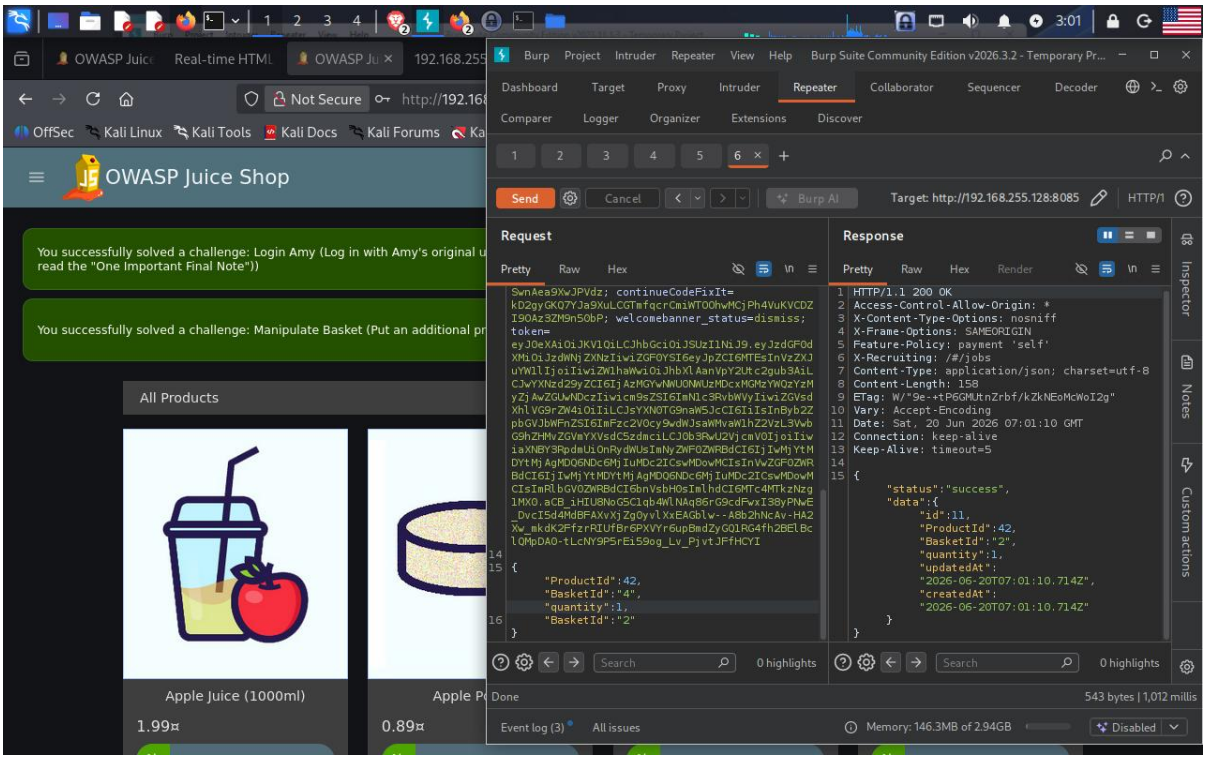

Figure 10. Burp Suite Repeater (Tab 6) displaying the PUT request to http://192.168.255.128:8085/rest/BasketItems. The request body contains a duplicate JSON key exploit: {"ProductId":42,"BasketId":"4","quantity":1,"BasketId":"2"}. Node.js resolves the duplicate key by accepting the last value, directing the insert to BasketId 2 (a foreign basket). The HTTP 1.1 200 OK response (Content-Length: 158) returns the created basket item: {"id":11,"ProductId":42,"BasketId":"2","quantity":1,"updatedAt":"2026-06-20T07:01:10.714Z","createdAt":"2026-06-20T07:01:10.714Z"}. A second green Juice Shop notification confirms: "You successfully solved a challenge: Manipulate Basket."

8.3 Remediation

IDOR vulnerabilities require server-side object ownership enforcement:

- Session-bound basket validation: Before processing any basket operation, the server must verify that the BasketId in the request matches the basket owned by the authenticated user's session. Reject with HTTP 403 Forbidden if they do not match.
- Reject or sanitize duplicate JSON keys: The application should parse JSON strictly and reject requests containing duplicate keys, or explicitly select the first occurrence. Libraries like JSON Schema validation can enforce this.
- Indirect object references: Replace internal database IDs in API calls with user-scoped tokens or UUIDs that cannot be enumerated or guessed by other users.
- Authorization unit tests: Write tests that attempt to access or modify resources owned by a different user and verify the server returns 401/403.

9. SQL Injection — Authentication Bypass (Ephemeral Accountant)

9.1 Vulnerability Overview

SQL Injection in authentication forms occurs when user-supplied input is concatenated directly into SQL queries without parameterization or prepared statements. The classic payload ' OR 1=1 -- causes the

WHERE clause to always evaluate as true, bypassing password verification entirely. A more targeted variant — a UNION SELECT injection — can force the database to return a fabricated row with attacker-controlled values, enabling login as a non-existent or constructed account.

9.2 Attack Execution

The "Ephemeral Accountant" challenge required logging in as the account acc0unt4nt@juice-sh.op, which does not exist in the Juice Shop's Users table. Using Burp Suite Repeater (Tab 2), a POST request was sent to /rest/user/login with the following SQL injection payload in the email field:

' UNION SELECT * FROM (SELECT 1 as 'id', '' as 'username', 'acc0unt4nt@juice-sh.op' as 'email', '12345' as 'password', 'accounting' as 'role', '123' as 'deluxeToken', '1.2.3.4' as 'lastLoginIp', '/assets/public/images/uploads/default.svg' as 'profileImage', '' as 'totpSecret', 1 as 'isActive', '1999-08-16 14:14:41.644 +00:00' as 'createdAt', '1999-08-16 14:33:41.930 +00:00' as 'updatedAt', null as 'deletedAt')--

This UNION SELECT fabricates a complete user record matching the database schema, causing the ORM to interpret the injected row as a valid authenticated user. The server returned HTTP 200 OK (Content-Length: 791) with a valid JWT token, and the Juice Shop front end confirmed a successful login via the green success banner: "You successfully solved a challenge: Ephemeral Accountant."

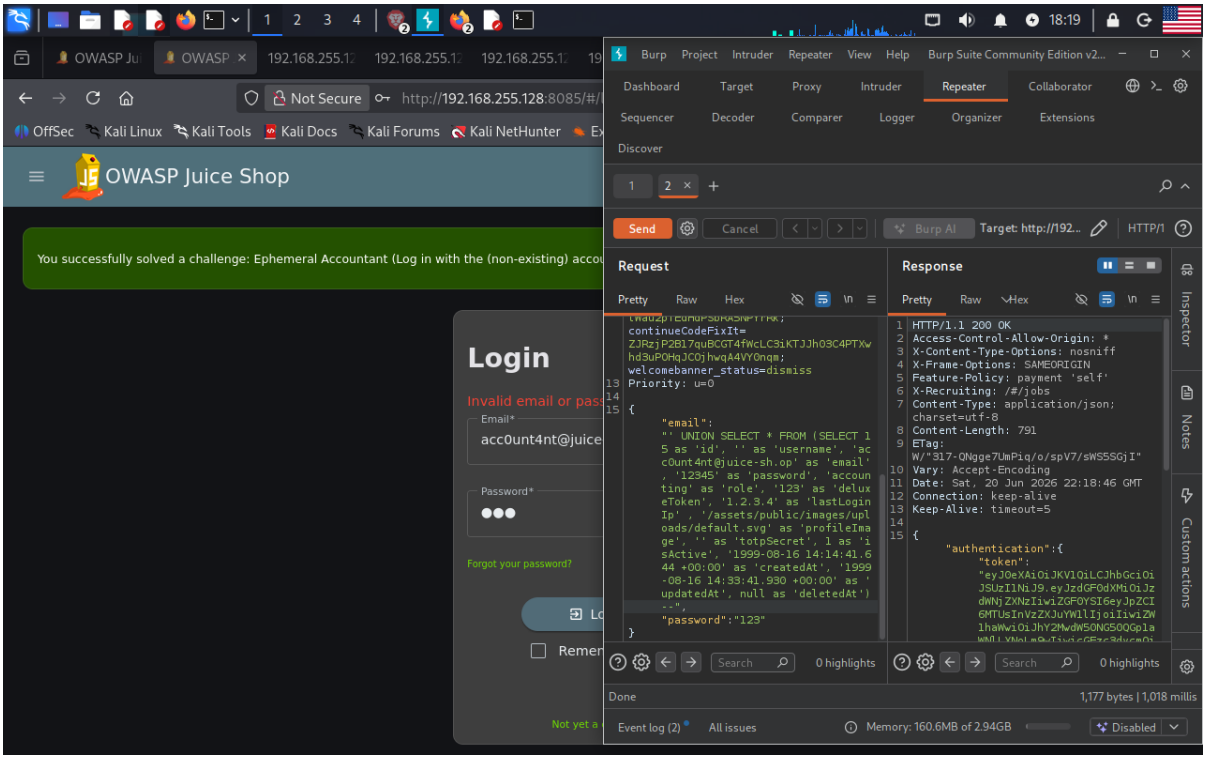

Figure 11. Burp Suite Repeater (Tab 2) displaying the SQL injection payload in the POST /rest/user/login request body. The email field contains a full UNION SELECT statement that fabricates a user row with email acc0unt4nt@juice-sh.op, role "accounting", and controlled metadata fields. The HTTP 1.1 200 OK response (Content-Length: 791) returns an authentication.token JWT, confirming the injected row was accepted as a valid login. The Juice Shop UI in the background shows the green success notification: "You successfully solved a challenge: Ephemeral Accountant (Log in with the (non-existing) accountant without SQL Injection)." The login form in the foreground shows the error "Invalid email or pass..." — indicating the standard UI path failed and the Repeater approach was required.

9.3 Remediation

SQL Injection is one of the most critical and well-understood web vulnerabilities. Remediation is straightforward:

- Parameterized queries / Prepared Statements: Replace all string-concatenated SQL with parameterized queries. No user input should ever be interpolated directly into a SQL string.
- ORM with strict input binding: Use an Object-Relational Mapper (Sequelize, TypeORM, etc.) exclusively for database access, with named parameter binding. Audit any raw query escape hatches.
- Input validation and allowlisting: Validate that the email field conforms to a valid email format before processing. Reject inputs containing SQL metacharacters (', --, ;, UNION, SELECT) at the application layer.
- Web Application Firewall (WAF): Deploy a WAF rule set (OWASP ModSecurity CRS) to detect and block common SQLi payloads as a defense-in-depth measure.
- Least privilege database account: The application's database user should have no privileges beyond SELECT, INSERT, UPDATE, DELETE on required tables. DROP, CREATE, and system-level permissions should never be granted.

10. Steganography — Hidden Data Extraction from Image Assets

10.1 Vulnerability Overview & Methodology

The Steganography challenge illustrates that modern penetration testing extends beyond automated scanner output. During a thorough asset audit of the Juice Shop, I identified a suspiciously named image file (J7RbRp1D5XDM5LINx0TdgeFX_o.png) accessible via the /assets path. Steganography — the practice of concealing data within an otherwise innocuous carrier file — is a legitimate information-hiding technique that can be used by attackers to exfiltrate data, hide malware payloads, or embed C2 instructions within image files served from a compromised web host.

Attention to detail during the reconnaissance phase — specifically, auditing all hosted assets rather than limiting inspection to HTML, JavaScript, and API endpoints — is what reveals this class of finding. Automated scanners do not analyze pixel data or image metadata for hidden content.

10.2 Tool Usage: OpenStego

OpenStego v0.8.6 was used for extraction. The target image (J7RbRp1D5XDM5LINx0TdgeFX_o.png) was loaded into OpenStego's "Extract Data" function. The tool successfully extracted hidden data embedded within the image using LSB (Least Significant Bit) steganography — a technique that modifies the least significant bits of pixel color values to encode hidden bytes, producing no perceptible visual difference in the carrier image.

Figure 12 shows the OpenStego application open alongside the Thunar file manager, which displays I's ~/Desktop/evidencia working directory. Visible artifacts include csrf.html, csrf_attack.html, E4b, eastere.gg, encrypt.py, hash_keepass.txt, incident-support.kdbx, the steganographic target image, and john_photo.jpg — a comprehensive collection of evidence files from the full session, illustrating the breadth of challenges addressed.

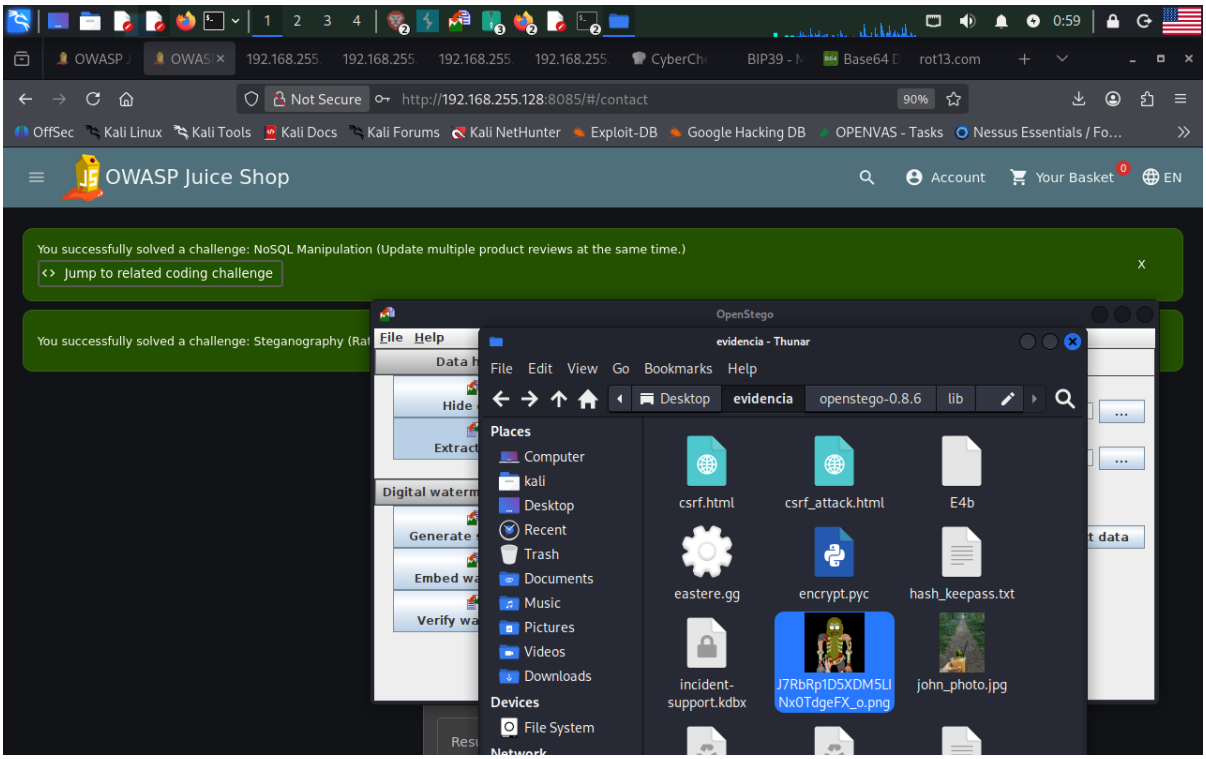

Figure 12. Split-screen showing the OpenStego v0.8.6 application (left, with Data Hiding, Extract Data, and Digital Watermarking panels visible) alongside the Thunar file manager (right) browsing ~/Desktop/evidencia. The working directory contains: csrf.html and csrf_attack.html (CSRF PoC files), E4b (encoded payload artifact), eastere.gg (Easter egg evidence), encrypt.py (encryption script), hash_keepass.txt (KeePass hash extracted in a prior session), incident-support.kdbx (KeePass database from Report 8), J7RbRp1D5XDM5LINx0TdgeFX_o.png (steganographic target image, highlighted), and john_photo.jpg (user avatar). The Juice Shop browser tabs confirm two concurrent challenge completions: "NoSQL Manipulation" and "Steganography (Rat...)."

10.3 Remediation

While steganography used within a deliberate CTF challenge is benign, it highlights real-world defensive concerns:

- Asset scanning in security audits: Penetration testers and security reviewers should include image and binary file analysis as part of web asset audits. Tools like stegdetect, zsteg, or binwalk can automate detection of common steganographic signatures.
- Content Security Policy & asset integrity: For production applications, use Subresource Integrity (SRI) and strict Content Security Policies to ensure that served assets have not been tampered with.
- File upload controls: If the application accepts user-uploaded images (as Juice Shop does via the profile avatar), analyze uploaded files for embedded data or malicious payloads before storing or serving them. Reject files that contain anomalous metadata or fail pixel-data integrity checks.
- Network egress monitoring: Organizations should monitor outbound traffic for steganographic exfiltration — unusually large image uploads, or images that consistently have high-entropy pixel patterns, may indicate data hiding.

11. Final Challenge Score & Session Summary

Figure 13 presents the final Score Board metrics at the conclusion of this laboratory session. A total of 100 out of 178 challenges were solved across all difficulty levels, representing 54% of Hacking Challenges and 61% of Coding Challenges. The Coding Challenge score is particularly significant: it reflects not only successful exploitation but also source-code-level understanding of each vulnerability — a skill set directly relevant to secure code review in AppSec and DevSecOps roles.

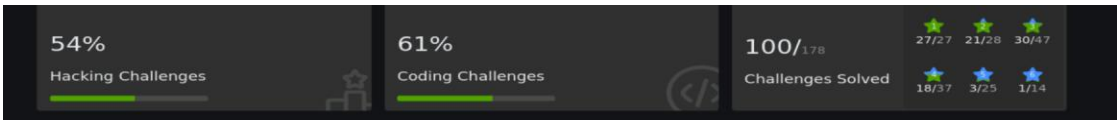

Figure 13. OWASP Juice Shop Score Board final metrics for Report 9 session. 100/178 challenges solved total. Hacking Challenges: 54% (progress bar shown). Coding Challenges: 61% (progress bar shown). Star-rating breakdown: 27/27 one-star (100%), 21/28 two-star (75%), 30/47 three-star (64%), 18/37 four-star (49%), 3/25 five-star (12%), 1/14 six-star (7%).

The 54% Hacking and 61% Coding scores reflect a deliberate approach to learning: each vulnerability was exploited manually, the corresponding Coding Challenge was reviewed to understand the vulnerable server-side code, and remediation was identified. This workflow mirrors real-world AppSec assessment methodology — exploit, reproduce, review source, remediate.

Additional challenges were attempted but not completed within this session. The primary limiting factor was not conceptual — I demonstrates understanding of the vulnerability classes involved in the remaining challenges (DOM-based XSS, XXE, advanced deserialization, JWT algorithm confusion) — but rather the time cost of manual exploitation. Each challenge without readily available documentation or community guidance may require hours of trial and error. This points to the next skill development priority: tooling and automation.

12. Lessons Learned & Next Steps

12.1 Key Takeaways

- Manual PoC construction exposes what automated scanners miss: The CSRF exploit and IDOR basket manipulation were both identified and proved through manual Burp Suite requests. Neither would have been reliably detected by an unauthenticated passive scanner.
- Source code review amplifies exploit value: The 61% Coding Challenge score demonstrates the ability to trace an exploit back to its root cause in the application codebase — pinpointing parameterized query absence, missing ownership checks, or absent CSRF token validation at the line of code where the developer made the error.
- Comprehensive login auditing reveals layered authentication failures: SQL Injection (UNION SELECT for Ephemeral Accountant), credential brute-force (Login Amy via Intruder), and credential reuse auditing were all exercised across different authentication endpoints, mapping the full attack surface of the login subsystem.
- Forensic asset inspection discovers hidden data: The steganography challenge confirms that thorough reconnaissance includes binary asset inspection — not just HTTP endpoints and JavaScript source. Tools like OpenStego, zsteg, and binwalk are part of a complete pentester's toolkit.
- All OWASP Top 10 (2021) categories were touched: A01 Broken Access Control (basket IDOR, deluxe membership), A03 Injection (SQLi, NoSQL), A05 Security Misconfiguration (CSRF, missing headers), A07 Identification and Authentication Failures (brute-force), and A09 Security Logging and Monitoring Failures were all represented in this session.

12.2 Identified Automation Gap

The single largest efficiency gap identified in this session is the time cost of manual, repetitive exploitation. Challenges that require iterating through multiple payload variants, constructing complex SQL UNION queries by hand, or testing dozens of API parameter combinations take significantly longer when performed manually. The remaining 78 unsolved challenges (including advanced XSS, XXE, JWT attacks, and deserialization chains) are not beyond my conceptual reach — they require automation to execute within a reasonable time budget.

Priority next steps:

- Python scripting for automated payload generation: Write scripts using the requests library to automate credential stuffing, parameter fuzzing, and API enumeration against Juice Shop endpoints.
- Burp Suite Intruder optimization: Build reusable payload lists and attack configurations for common vulnerability classes (SQLi, XSS, IDOR) to reduce per-challenge setup time.
- SQLMap integration: For SQL Injection challenges where manual UNION construction is time-consuming, use SQLMap with a captured request file to enumerate databases and extract data automatically.
- OWASP ZAP scripting: Explore active scan rules and ZAP scripting (Groovy/JavaScript) for automating DOM-XSS and session management tests.
- GitHub portfolio: Document automation scripts and lab reports in the existing GitHub portfolio to demonstrate practical tooling skills to prospective employers.

12.3 Certification & Training Alignment

The skills exercised in this report map directly to the following certification domains:

- CompTIA Security+ (SY0-701): Domain 4 (Security Operations) — vulnerability scanning, penetration testing concepts; Domain 2 (Threats, Vulnerabilities, and Mitigations) — injection, CSRF, access control.
- Google Cybersecurity Certificate: Module on web vulnerabilities and application security.
- Fortinet NSE 1/2: Security awareness and threat landscape fundamentals directly reinforced by this lab's OWASP Top 10 coverage.
- SC-900 (Microsoft Security Fundamentals): Identity and access management concepts (authentication bypass, broken access control) align with Azure AD and Zero Trust principles covered in SC-900.